UNIVERSIDAD MAYOR DE SAN ANDRES
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA

MÉTODOS NUMÉRICOS

Tarea 1

Fundamentación Teórica de Métodos Iterativos

Métodos Analizados:

- Método de Gauss-Seidel
- Método de Sobre-relajación Sucesiva (SOR)
- Método de Gradiente Conjugado (CG)

Estudiante: IVER SAMUEL MEDINA BALBOA
Docente: LIC. BRIGIDA CARVAJAL BLANCO
Materia: Métodos Numéricos I
Fecha: 6 de abril de 2026

Índice general

1. Introducción **2**

2. Sistemas de ecuaciones lineales **3**

3. Método de Gauss-Seidel **4**
3.1. Descripción del método 4
3.2. Algoritmo 4
3.3. Condiciones de convergencia 4
3.4. Características 5
3.4.1. Propiedades 6
3.4.2. Aplicaciones 6
3.4.3. Complejidad Computacional 6
3.4.4. Aplicaciones 6
3.5. Representación gráfica 6

4. Método de Sobre-relajación Sucesiva (SOR) **7**
4.1. Descripción 7
4.2. Interpretación 7
4.3. Algoritmo 7
4.4. Condiciones de convergencia 8
4.5. Ventajas 8
4.6. Desventajas 8
4.6.1. Elección del Parámetro 8
4.6.2. Aplicaciones 8

5. Método del Gradiente Conjugado **10**
5.1. Descripción 10
5.2. Idea geométrica 10
5.3. Algoritmo 10
5.4. Condiciones de convergencia 10
5.5. Características 10

6. Comparación de los métodos **12**
6.0.1. Comparación Matemática 12
6.0.2. Ejemplo Numérico 12

7. Conclusiones **14**

Capítulo 1

Introducción

En el área de métodos numéricos, los sistemas de ecuaciones lineales aparecen en múltiples aplicaciones científicas y de ingeniería, tales como simulaciones físicas, optimización, análisis estructural y resolución de ecuaciones diferenciales parciales.

Cuando el tamaño del sistema es pequeño, los métodos directos como eliminación de Gauss o factorización LU son adecuados. Sin embargo, cuando los sistemas se vuelven muy grandes, estos métodos pueden resultar computacionalmente costosos en términos de memoria y tiempo.

En estos casos, se utilizan métodos iterativos, los cuales generan una sucesión de aproximaciones que convergen a la solución exacta del sistema. Entre los métodos iterativos más importantes se encuentran:

- Método de Gauss-Seidel
- Método de Sobre-relajación Sucesiva (SOR)
- Método de Gradiente Conjugado

Estos métodos permiten resolver sistemas lineales grandes de forma eficiente y son ampliamente utilizados en informática científica y computación de alto rendimiento.

Capítulo 2

Sistemas de ecuaciones lineales

Un sistema de ecuaciones lineales puede escribirse en forma matricial como:

$$Ax = b$$

donde:

- A es una matriz de coeficientes de tamaño $n \times n$
- x es el vector de incógnitas
- b es el vector de términos independientes

Resolver el sistema consiste en encontrar el vector x que satisface la ecuación.

En los métodos iterativos, se comienza con una aproximación inicial $x^{(0)}$ y se genera una secuencia:

$$x^{(1)}, x^{(2)}, x^{(3)}, ...$$

hasta que se cumple un criterio de convergencia.

Capítulo 3

Método de Gauss-Seidel

3.1. Descripción del método

El método de Gauss-Seidel es un método iterativo utilizado para resolver sistemas de ecuaciones lineales. Es una mejora del método de Jacobi, ya que utiliza inmediatamente los valores actualizados durante el proceso iterativo.

El sistema

$$Ax = b$$

se descompone en:

$$A = D + L + U$$

donde:

- D es la matriz diagonal
- L es la matriz triangular inferior
- U es la matriz triangular superior

La fórmula iterativa es:

$$x^{(k+1)} = (D + L)^{-1}(b - Ux^{(k)})$$

3.2. Algoritmo

3.3. Condiciones de convergencia

El método converge si:

- La matriz es diagonalmente dominante
- La matriz es simétrica definida positiva

Algorithm 1 Método de Gauss-Seidel

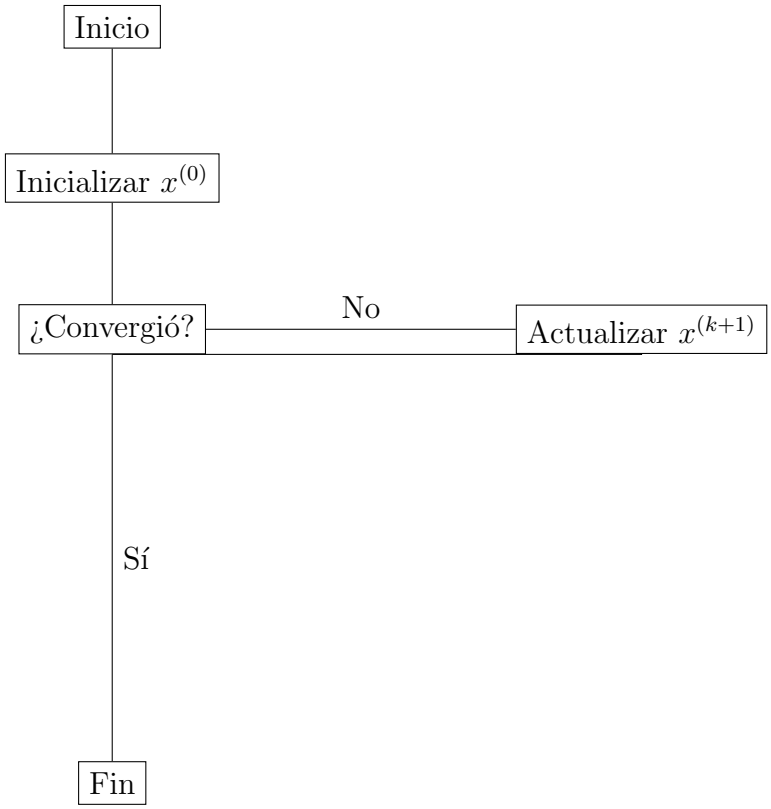
Elegir aproximación inicial $x^{(0)}$
for $k = 0, 1, 2, ...$ **do**
  for $i = 1$ hasta n **do**

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right)$$

  end for
end for

Una matriz es diagonalmente dominante si:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$



3.4. Características

- Método iterativo simple
- Requiere poca memoria
- Convergencia más rápida que Jacobi
- Puede divergir si la matriz no cumple condiciones

3.4.1. Propiedades

- Utiliza los valores más recientes en cada paso.
- Generalmente converge más rápido que Jacobi.
- Puede ser lento si la matriz no es bien condicionada.

3.4.2. Aplicaciones

- Resolución de sistemas en métodos numéricos básicos.
- Problemas de ingeniería eléctrica.
- Modelado de ecuaciones diferenciales discretizadas.

3.4.3. Complejidad Computacional

Cada iteración requiere:

- Un producto matriz-vector Ap_k.
- Dos productos internos.
- Actualización de vectores.

Esto lo hace eficiente para matrices dispersas, con complejidad aproximada $O(n)$ por iteración.

3.4.4. Aplicaciones

- Métodos de elementos finitos en ingeniería.
- Optimización en aprendizaje automático.
- Simulación de sistemas físicos.

3.5. Representación gráfica

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right)$$

Figura 3.1: Proceso iterativo del método de Gauss-Seidel

Capítulo 4

Método de Sobre-relajación Sucesiva (SOR)

4.1. Descripción

El método SOR es una extensión del método de Gauss-Seidel que introduce un factor de relajación ω para acelerar la convergencia.

La fórmula iterativa es:

$$x_i^{(k+1)} = (1-\omega)x_i^{(k)} + \frac{\omega}{a_{ii}}\left(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)}\right)$$

donde ω es el factor de relajación.

4.2. Interpretación

- $\omega = 1 \rightarrow$ Gauss-Seidel
- $0 < \omega < 1 \rightarrow$ sub-relajación
- $1 < \omega < 2 \rightarrow$ sobre-relajación

4.3. Algoritmo

Algorithm 2 Método SOR

```
Elegir x^(0)
Elegir ω
for k = 0, 1, 2, ... do
    for i = 1 hasta n do
      Actualizar x_i
    end for
end for
```

4.4. Condiciones de convergencia

El método converge si:

- $0 < \omega < 2$
- la matriz es definida positiva

4.5. Ventajas

- Más rápido que Gauss-Seidel
- Muy utilizado en simulaciones físicas

4.6. Desventajas

- Elegir ω óptimo puede ser complicado

4.6.1. Elección del Parámetro

- Si $\omega = 1$, se obtiene Gauss-Seidel.
- Si $0 < \omega < 1$, se habla de sub-relajación.
- Si $1 < \omega < 2$, se puede acelerar la convergencia.

4.6.2. Aplicaciones

- Problemas de dinámica de fluidos.
- Simulación de procesos térmicos.
- Resolución de sistemas grandes en ingeniería civil y mecánica.

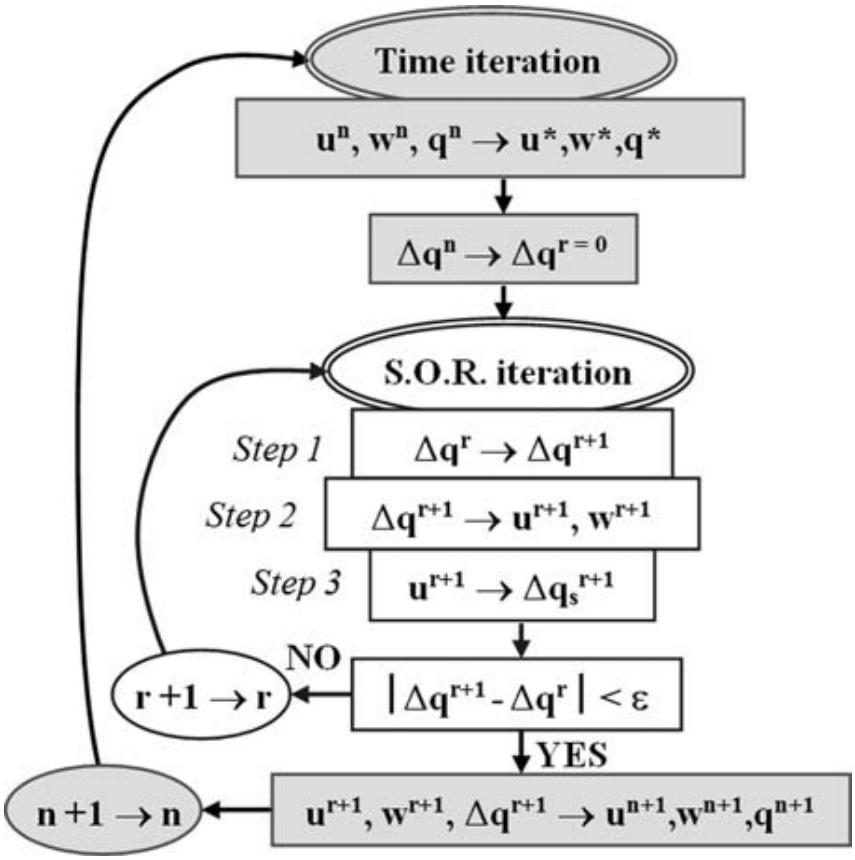


Figura 4.1: Comparación de convergencia entre Jacobi, Gauss-Seidel y SOR

Capítulo 5

Método del Gradiente Conjugado

5.1. Descripción

El método del Gradiente Conjugado es un algoritmo iterativo utilizado para resolver sistemas de ecuaciones lineales donde la matriz es simétrica y definida positiva.

Fue desarrollado por Hestenes y Stiefel en 1952 y es uno de los métodos más importantes en computación científica.

5.2. Idea geométrica

El método busca minimizar la función cuadrática:

$$f(x) = \frac{1}{2}x^T Ax - b^T x$$

mediante direcciones conjugadas.

5.3. Algoritmo

5.4. Condiciones de convergencia

- La matriz debe ser simétrica
- Debe ser definida positiva

5.5. Características

- Muy eficiente para matrices grandes
- No requiere almacenar la matriz completa
- Convergencia rápida

Algorithm 3 Gradiente Conjugado

Elegir x_0
$r_0 = b - Ax_0$
$p_0 = r_0$
for $k = 0, 1, \ldots$ **do**

$$\alpha_k = \frac{r_k^T r_k}{p_k^T A p_k}$$

$$x_{k+1} = x_k + \alpha_k p_k$$

$$r_{k+1} = r_k - \alpha_k A p_k$$

$$\beta_k = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$$

$$p_{k+1} = r_{k+1} + \beta_k p_k$$

end for

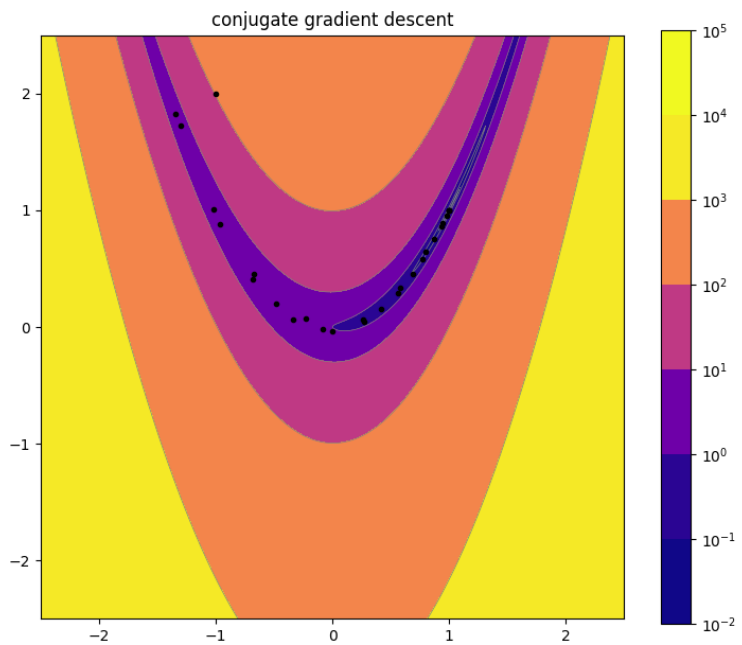


Figura 5.1: Interpretación geométrica del método de gradiente conjugado

Capítulo 6

Comparación de los métodos

6.0.1. Comparación Matemática

Podemos expresar los tres métodos en forma matricial:

- **CG:** Minimización de $f(x)$ con direcciones conjugadas.
- **Gauss-Seidel:** Iteración $x^{(k+1)} = (D + L)^{-1}(b - Ux^{(k)})$.
- **SOR:** Iteración $x^{(k+1)} = (1 - \omega)x^{(k)} + \omega(D + L)^{-1}(b - Ux^{(k)})$.

La matriz de iteración determina la convergencia:

$$x^{(k+1)} = Mx^{(k)} + c$$

donde M depende del método. La condición de convergencia es:

$$\rho(M) < 1$$

siendo $\rho(M)$ el radio espectral.

Método	Tipo	Velocidad	Uso
Gauss-Seidel	Iterativo	Media	Sistemas moderados
SOR	Iterativo	Alta	Simulación numérica
Gradiente Conjugado	Iterativo	Muy alta	Sistemas grandes

6.0.2. Ejemplo Numérico

Comparación con Gauss-Seidel para $\omega = 1{,}2$ muestra aceleración en la convergencia.

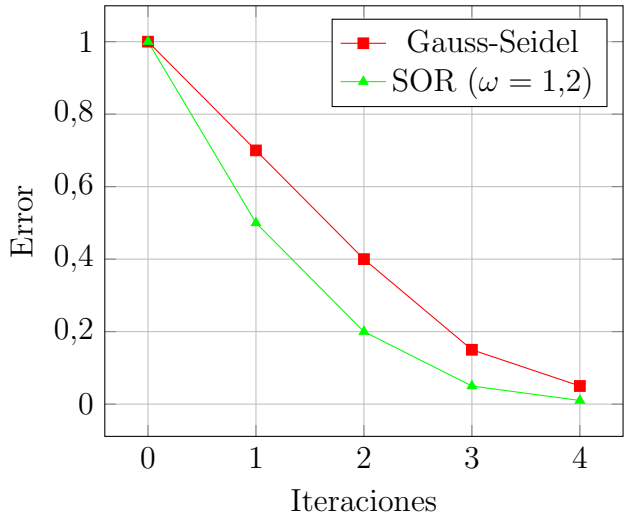


Figura 6.1: Comparación de convergencia entre Gauss-Seidel y SOR

Tarea 2

Para esta etapa se plantea una sistema de ecuaciones lineales con solucion exacta. La misma se presenta a continuación:

$$\begin{cases} 10x + 2y + 3z = 15 \\ 2x + 10y + z = 13 \\ 3x + y + 10z = 14 \end{cases} \tag{1}$$

También se puede expresar de la forma matricial:

$$A = \begin{bmatrix} 10 & 2 & 3 \\ 2 & 10 & 1 \\ 3 & 1 & 10 \end{bmatrix}$$

$$x = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$b = \begin{bmatrix} 15 \\ 13 \\ 14 \end{bmatrix}$$

Para la correcta implementación de los metodos, es pertinente verificar la predominancia de la diagonal, es decir, la matriz sea **diagonalmente dominante**, por lo tanto se tiene:

$$|10| > |2| + |3|$$

$$|10| > |2| + |1|$$

$$|10| > |3| + |1|$$

Por lo tanto, podemos concluir que la matriz A es **diagonalmente dominante**.
Para corroborar lo sindicado lineas arriba, se procederá a resolver el sistema de manera. analítica.
Por otro lado, tambien podemos corroborar que la matriz es simetrica, lo que permite la resolucion por el metodo Gradiente Conjugado
Despejando *x, y, z* de las tres ecuaciones respectivamente, tenemos:

$$x = \frac{1}{10}(15 - 2y - 3z)$$

$$y = \frac{1}{10}(13 - 2x - z)$$

$$z = \frac{1}{10}(14 - 3x - y)$$

Resolviendo, la solución exacta para *x, y, z* convergen en *(1,1,1)*
Se ha demostrado que el sistema lineal 3 x 3 presenta solución exacta.

Tarea 3

Para la resolucion del sistema de ecuaciones aplicando los tres metodos, se aplico del programa *Python*, donde a continuacion se comparte el codigo y la salida de cada ejecución.

```python
import numpy as np
#Ecuaciones lineales en su forma matricial de la forma Ax=b
A = np.array([[10,2,3],[2,10,1],[3,1,10]], dtype = float)
b = np.array([15,13,14], dtype=float)
#Valor inicial de ceros para el metodo
x0 = np.zeros(3)

# Aplicacion del Metodo de Gauss Seidel

def met_gauss_Seidel(A,b,x, it=20):
    n = len(b)
    x=x.copy()

    print(f'Metodo de Gauss Seidel con {it} iteraciones')
    for k in range(it):
        for i in range(n):
            s1=sum(A[i,j]*x[j] for j in range(i))
            s2=sum(A[i,j]*x[j] for j in range(i+1,n))

            x[i]=(b[i]-s1-s2)/A[i,i]
        print(f'Iteraciones: {k+1} -> {x}')

met_gauss_Seidel(A,b,x0)

# Aplicacion del Metodo de SOR

def met_SOR(A,b,x,w=1.1,it=15):
    n = len(b)
    x = x.copy()

    print(f'Metodo de SOR con {it} iteraciones y omega = {w}')

    for k in range(it):
        for i in range(n):
            s1=sum(A[i,j]*x[j] for j in range(i))
            s2=sum(A[i,j]*x[j] for j in range (i+1,n))

            xi=(b[i]-s1-s2)/A[i,i]

            x[i]=(1-w)*x[i] + w*xi
        print(f'Iteraciones: {k+1} -> {x}')
met_SOR(A,b,x0)

# Aplicacion del metodo de gradiente conjugado

def met_gradiente_conjugado(A,b,x,it=10):
    x = x.copy()

    r= b - A@x
    p = r.copy()

    print(f'Metodo de Gradiente Conjugado con {it} Iteraciones')
```

```python
    for k in range(it):
        Ap = A @ p
        alpha = (r @ r) / (p @ Ap)
        x = x + alpha*p
        r2 = r - alpha*Ap
        print(f'Iteraciones : {k+1} -> {x}')

        if np.linalg.norm(r2) < 1e-10:
            break
        beta = (r2 @ r2) / (r @ r)
        p = r2 + beta*p
        r = r2
    return x
met_gradiente_conjugado(A,b,x0)
```

A continuación, se presenta la pantalla de ejecución:

Método de Gauss-Seidel

```
Metodo de Gauss Seidel con 15 iteraciones
Iteraciones: 1 -> [1.5  1.   0.85]
Iteraciones: 2 -> [1.045  1.006  0.9859]
Iteraciones: 3 -> [1.00303   1.000804  0.9990106]
Iteraciones: 4 -> [1.00013602 1.00007174 0.99995202]
Iteraciones: 5 -> [1.00000005 1.00000479 0.99999951]
Iteraciones: 6 -> [0.99999919 1.00000021 1.00000022]
Iteraciones: 7 -> [0.99999989 1.         1.00000003]
Iteraciones: 8 -> [0.99999999 1.         1.        ]
Iteraciones: 9 -> [1. 1. 1.]
Iteraciones: 10 -> [1. 1. 1.]
Iteraciones: 11 -> [1. 1. 1.]
Iteraciones: 12 -> [1. 1. 1.]
Iteraciones: 13 -> [1. 1. 1.]
Iteraciones: 14 -> [1. 1. 1.]
Iteraciones: 15 -> [1. 1. 1.]
```

Método de SOR

```
Metodo de SOR con 15 iteraciones y omega = 1.1
Iteraciones: 1 -> [1.65    1.067   0.87813]
Iteraciones: 2 -> [0.9604771  1.01540074 1.02353548]
Iteraciones: 3 -> [0.99279742 0.99745559 1.00030319]
Iteraciones: 4 -> [1.00117998 0.9999615  0.99958452]
Iteraciones: 5 -> [1.00002758 1.00004349 1.00002766]
Iteraciones: 6 -> [0.99997855 0.99999733 1.00000461]
Iteraciones: 7 -> [1.00000121 0.99999949 0.99999919]
Iteraciones: 8 -> [1.00000026 1.00000008 0.99999999]
Iteraciones: 9 -> [0.99999996 1.         1.00000001]
Iteraciones: 10 -> [1. 1. 1.]
Iteraciones: 11 -> [1. 1. 1.]
Iteraciones: 12 -> [1. 1. 1.]
Iteraciones: 13 -> [1. 1. 1.]
Iteraciones: 14 -> [1. 1. 1.]
Iteraciones: 15 -> [1. 1. 1.]
```

Método de Gradiente Conjugado

```
Metodo de Gradiente Conjugado con 15 Iteraciones
Iteraciones : 1 -> [1.06575145 0.92365125 0.99470135]
Iteraciones : 2 -> [1.00842018 1.00273043 0.98832982]
Iteraciones : 3 -> [1. 1. 1.]
```

Se realizara una comparación de cada método con la solución exacta Sabemos que:

$$(x, y, z) = (1, 1, 1)$$

El error absoluto se define de la forma:

$$Error = |X - Xexacto|$$

Método de Gauss-Seidel

Variable	**Aproximación**	**Exacta**	**Error**
x	1	1	0
y	1	2	0
z	1	3	0

Método de SOR

Variable	**Aproximación**	**Exacta**	**Error**
x	1	1	0
y	1	2	0
z	1	3	0

Método de Gradiente Conjugado

Variable	**Aproximación**	**Exacta**	**Error**
x	1	1	0
y	1	2	0
z	1	3	0

Realizando una comparacion final entre los 3 métodos, obtenemos

Método	**Iteraciones**	**Resultado Aproximado**
Gauss-Seidel	9	$\mathbf{x} \approx (1{,}0, 2{,}0, 3{,}0)$
SOR ($\omega = 1$)	10	$\mathbf{x} \approx (1{,}0, 2{,}0, 3{,}0)$
Gradiente Conjugado	3	$\mathbf{x} \approx (1{,}0, 2{,}0, 3{,}0)$

Tarea 4

Para la resolución de la tarea 4, se comparte el documento Excel, donde se detalla la resolución del sistema de ecuaciones por cada metodo.

Tarea 5

Para el presente apartado se plantea un problema de aplicación real orientado a la energía fotovoltaica. Se realiza una breve descripción del problema, modelación y obtención de la solución exacta.
Como punto inicial, el sistema que se plantea es el estudio del comportamiento de una **micro-red fotovoltaica distribuida**, donde múltiples nodos intercambian energía bajo condiciones de equilibrio. Todo el sistema se basa en el principio básico de la *conservación de la energía*.
Se considera una microrred como un sistema eléctrico integrado formado por generación y almacenamiento distribuidos y cargas eléctricas múltiples que actúan como una sola **red**. Esta microrred puede ser formada por muchas combinaciones de generación y almacenamiento y formatos de conectividad de red. Las microrredes integran asimismo fuentes de energía renovable (fotovoltaica, como en el presente caso).
Un sistema de control de microrred realiza un control dinámico de las fuentes de energía y permite operaciones autónomas y automáticas de autorreparacion.

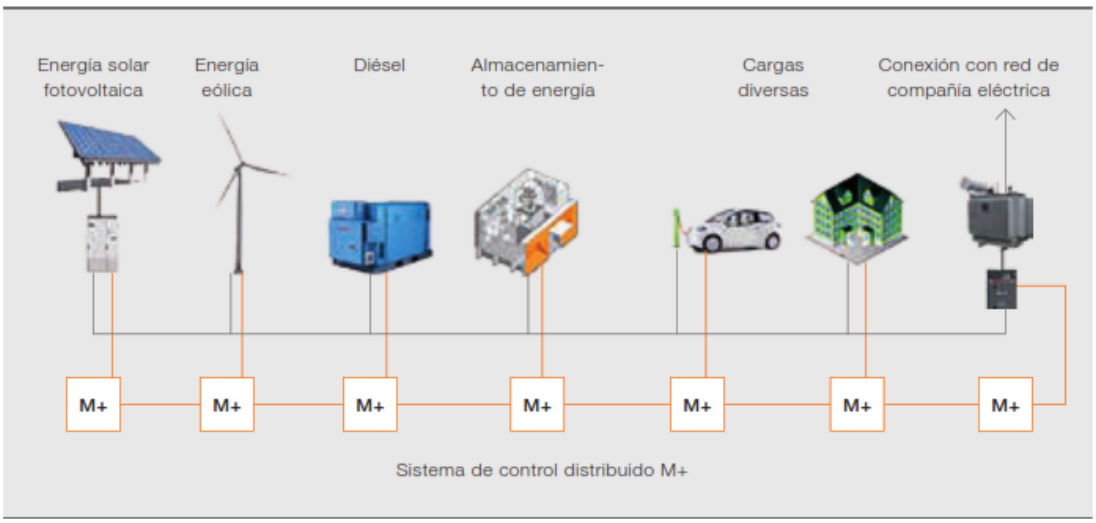


Figura 1: Sistema de control

Como se evidencia en la figura 1, se puede evidencia un sistema clasico de **microrred**, donde se evidencia las fuentes de generación, las fuentes de almacenamiento y su distribucion.
En sintesis, podemos resumir una **microrred** de la forma:

- *Generación: Fuentes renovables (fotovoltaica)*
- *Almacenamient: Electrico*
- *Cargas*
- *Sistema de control*

Para el presente apartado, como se indico lineas arriba, se presenta el modelado del sistema presentado a continuación:

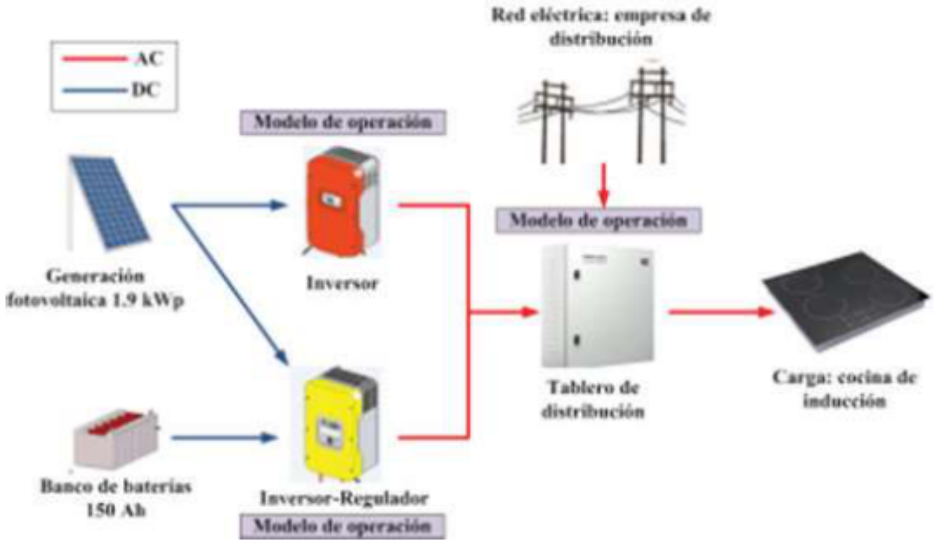


Figura 2: Sistema propuesto

Teóricamente, las ecuaciones que se presentan a continuación se obtienen a partir de la ley de balance energético:

$$E_(generada) = E_(consumida) + E_(transferida) + pérdidas$$

Este principio básico permite modelar matematicamente la interacción entre los distintos componentes del sistema, de la forma:

$$Ax = b$$

Donde: A = Coeficientes de interaccion entre nodos;
x = Vector de las variables energeticas;
b = demandas o suministros externos del sistema
Sea:

$$\begin{cases} 10x_1 - 2x_2 - x_3 = 7 \\ -2x_1 + 10x_2 - 2x_3 - x_4 = 8 \\ -x_1 - 2x_2 + 10x_3 - 2x_4 - x_5 = 6 \\ -x_2 - 2x_3 + 10x_4 - 2x_5 = 5 \\ -x_3 - 2x_4 + 10x_5 = 4 \end{cases} \tag{1}$$

Este modelo es una **microrred fotovoltaica** con 5 nodos interconectados.
Donde:
x1 = Energia generada por el panel 1;
x2 = Energia generada por el panel 2;
x3 = Energia almacenada en la bateria;
x4 = Energia suministrada a la carga residencial;
x5 = Energia suministrada a la carga industrial;
Por ejemplo:

$$10x_1 - 2x_2 - x_3 = 7$$

Donde:
10x1 : produccion del panel 1;

-*2x2* : interaccion con el panel 2, es decir, el signo negativo significa transferencia;
-*x3* : energia enviada a la bateria;
7 : energia requerida o entregada al sistema.
También se puede expresar de la forma matricial:

$$Ax = b$$

Entonces:

$$A = \begin{bmatrix} 10 & -2 & -1 & 0 & 0 \\ -2 & 10 & -2 & -1 & 0 \\ -1 & -2 & 10 & -2 & -1 \\ 0 & -1 & -2 & 10 & -2 \\ 0 & 0 & -1 & -2 & 10 \end{bmatrix}$$

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix}$$

$$b = \begin{bmatrix} 7 \\ 8 \\ 6 \\ 5 \\ 4 \end{bmatrix}$$

Para la correcta implementación de los metodos, es pertinente verificar la predominancia de la diagonal, es decir, la matriz sea **diagonalmente dominante**, por lo tanto se tiene:

$$|10| > |2| + |1|$$

$$|10| > |2| + |2| + |1|$$

$$|10| > |1| + |2| + |1| + |2|$$

$$|10| > |2| + |1| + |2|$$

$$|10| > |1| + |1|$$

Por lo tanto, podemos concluir que la matriz A es **diagonalmente dominante**.
Para corroborar lo sindicado lineas arriba, se procederá a resolver el sistema de manera. analítica.
Por otro lado, también podemos corroborar que la matriz es simétrica, lo que permite la resolución por el método **Gradiente Conjugado**
Resolviendo
Despejamos x_1 de la primera ecuación

$$10x_1 - 2x_2 - x_3 = 7 \quad \Rightarrow \quad x_1 = \frac{7 + 2x_2 + x_3}{10}$$

—

Sustituimos en la segunda ecuación

$$-2x_1 + 10x_2 - 2x_3 - x_4 = 8$$

$$-2\left(\frac{7+2x_2+x_3}{10}\right)+10x_2-2x_3-x_4=8$$

$$-\frac{14}{10}-\frac{4}{10}x_2-\frac{2}{10}x_3+10x_2-2x_3-x_4=8$$

$$-1{,}4-0{,}4x_2-0{,}2x_3+10x_2-2x_3-x_4=8$$

$$9{,}6x_2-2{,}2x_3-x_4=9{,}4$$

—

Tercera ecuación

$$-x_1-2x_2+10x_3-2x_4-x_5=6$$

$$-\frac{7+2x_2+x_3}{10}-2x_2+10x_3-2x_4-x_5=6$$

$$-0{,}7-0{,}2x_2-0{,}1x_3-2x_2+10x_3-2x_4-x_5=6$$

$$-2{,}2x_2+9{,}9x_3-2x_4-x_5=6{,}7$$

—

Cuarta ecuación

$$-x_2-2x_3+10x_4-2x_5=5$$

—

Quinta ecuación

$$-x_3-2x_4+10x_5=4$$

—

Sistema reducido en cuatro variables

$$\begin{gathered}9{,}6x_2-2{,}2x_3-x_4=9{,}4\\-2{,}2x_2+9{,}9x_3-2x_4-x_5=6{,}7\\-x_2-2x_3+10x_4-2x_5=5\\-x_3-2x_4+10x_5=4\end{gathered}$$

Resolviendo este sistema se obtiene:

$$x_2=1,\quad x_3=1,\quad x_4=1,\quad x_5=1$$

—

Sustituimos en x_1

$$x_1=\frac{7+2(1)+1}{10}=\frac{10}{10}=1$$

—

Solución final

$$x_1 = 1, \quad x_2 = 1, \quad x_3 = 1, \quad x_4 = 1, \quad x_5 = 1$$

La solución exacta para *x1, x2, x3, x4, x5* convergen en *(1,1,1,1,1)*

Se ha demostrado que el sistema lineal 5 x 5 presenta solución exacta y única.

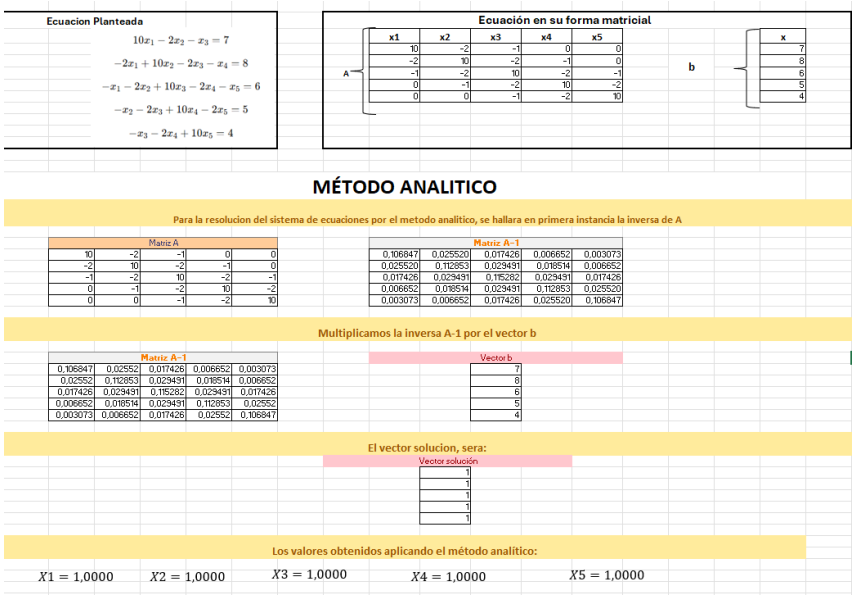
Tarea 6

Se comparte la resolucion de este problema planteado en:

- Excel;
- Python;
- Web Interactiva desarrollada con PHP.

Excel

Para este apartado, se ha utilizado un archivo Excel, separado por pestañas donde se presenta la solución única, por lo métodos: Gauss-Seidel, SOR y Gradiente Conjugado.



Ecuacion Planteada

$$10x_1 - 2x_2 - x_3 = 7$$
$$-2x_1 + 10x_2 - 2x_3 - x_4 = 8$$
$$-x_1 - 2x_2 + 10x_3 - 2x_4 - x_5 = 6$$
$$-x_2 - 2x_3 + 10x_4 - 2x_5 = 5$$
$$-x_3 - 2x_4 + 10x_5 = 4$$

Ecuación en su forma matricial

A

x1	x2	x3	x4	x5
10	-2	-1	0	0
-2	10	-2	-1	0
-1	-2	10	-2	-1
0	-1	-2	10	-2
0	0	-1	-2	10

b

x
7
8
6
5
4

MÉTODO ANALITICO

Para la resolucion del sistema de ecuaciones por el metodo analitico, se hallara en primera instancia la inversa de A

Matriz A

10	-2	-1	0	0
-2	10	-2	-1	0
-1	-2	10	-2	-1
0	-1	-2	10	-2
0	0	-1	-2	10

Matriz A-1

0,106847	0,025520	0,017426	0,006652	0,003073
0,025520	0,112853	0,029491	0,018514	0,006652
0,017426	0,029491	0,115282	0,029491	0,017426
0,006652	0,018514	0,029491	0,112853	0,025520
0,003073	0,006652	0,017426	0,025520	0,106847

Multiplicamos la inversa A-1 por el vector b

Matriz A-1

0,106847	0,02552	0,017426	0,006652	0,003073
0,02552	0,112853	0,029491	0,018514	0,006652
0,017426	0,029491	0,115282	0,029491	0,017426
0,006652	0,018514	0,029491	0,112853	0,02552
0,003073	0,006652	0,017426	0,02552	0,106847

Vector b

7
8
6
5
4

El vector solucion, sera:

Vector solución

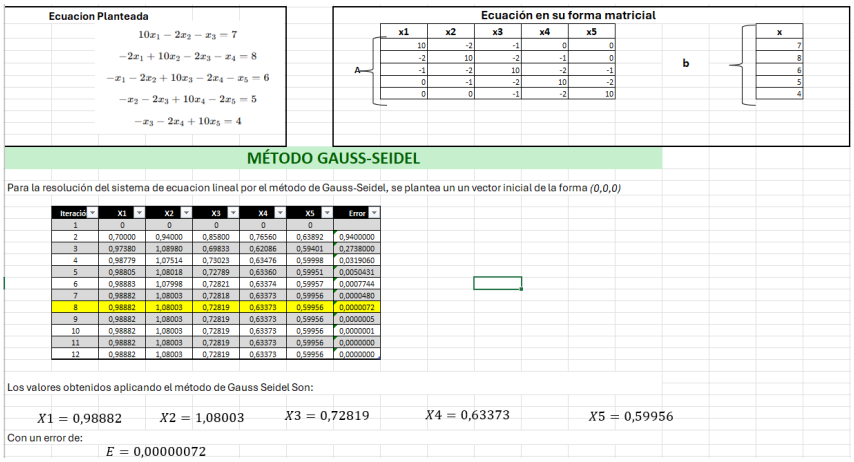
1
1
1
1
1

Los valores obtenidos aplicando el método analitico:

$X1 = 1{,}0000$ $X2 = 1{,}0000$ $X3 = 1{,}0000$ $X4 = 1{,}0000$ $X5 = 1{,}0000$

Figura 3: Solucion Unica

Por los metodos planteados:



Ecuacion Planteada

$$10x_1 - 2x_2 - x_3 = 7$$
$$-2x_1 + 10x_2 - 2x_3 - x_4 = 8$$
$$-x_1 - 2x_2 + 10x_3 - 2x_4 - x_5 = 6$$
$$-x_2 - 2x_3 + 10x_4 - 2x_5 = 5$$
$$-x_3 - 2x_4 + 10x_5 = 4$$

Ecuación en su forma matricial

A

x1	x2	x3	x4	x5
10	-2	-1	0	0
-2	10	-2	-1	0
-1	-2	10	-2	-1
0	-1	-2	10	-2
0	0	-1	-2	10

b

x
7
8
6
5
4

MÉTODO GAUSS-SEIDEL

Para la resolución del sistema de ecuacion lineal por el método de Gauss-Seidel, se plantea un un vector inicial de la forma *(0,0,0)*

Iteración	X1	X2	X3	X4	X5	Error
1	0	0	0	0	0	
2	0,70000	0,94000	0,85800	0,76560	0,63892	0,9400000
3	0,97380	1,08980	0,69833	0,62086	0,59401	0,2738000
4	0,98779	1,07514	0,73023	0,63476	0,59998	0,0319060
5	0,98805	1,08018	0,72789	0,63360	0,59951	0,0050431
6	0,98883	1,07998	0,72821	0,63374	0,59957	0,0007744
7	0,98882	1,08003	0,72818	0,63373	0,59956	0,0000480
8	0,98882	1,08003	0,72819	0,63373	0,59956	0,0000072
9	0,98882	1,08003	0,72819	0,63373	0,59956	0,0000005
10	0,98882	1,08003	0,72819	0,63373	0,59956	0,0000001
11	0,98882	1,08003	0,72819	0,63373	0,59956	0,0000000
12	0,98882	1,08003	0,72819	0,63373	0,59956	0,0000000

Los valores obtenidos aplicando el método de Gauss Seidel Son:

$X1 = 0{,}98882$ $X2 = 1{,}08003$ $X3 = 0{,}72819$ $X4 = 0{,}63373$ $X5 = 0{,}59956$

Con un error de:

$E = 0{,}00000072$

Figura 4: Metodo Gauss Seidel

Tambien, los métodos de SOR y Gradiente Conjugado

Ecuacion Planteada

$$10x_1 - 2x_2 - x_3 = 7$$
$$-2x_1 + 10x_2 - 2x_3 - x_4 = 8$$
$$-x_1 - 2x_2 + 10x_3 - 2x_4 - x_5 = 6$$
$$-x_2 - 2x_3 + 10x_4 - 2x_5 = 5$$
$$-x_3 - 2x_4 + 10x_5 = 4$$

Ecuación en su forma matricial

A

x1	x2	x3	x4	x5
10	-2	-1	0	0
-2	10	-2	-1	0
-1	-2	10	-2	-1
0	-1	-2	10	-2
0	0	-1	-2	10

b

x
7
8
6
5
4

w	1,1

MÉTODO SOR

Para la resolución del sistema de ecuacion lineal por el método SOR, se plantea un un vector inicial de la forma *(0,0,0)*

Tomando como valor para Omega $\omega = 1{,}1$

Iteraci	X1	X2	X3	X4	X5	Error
1	0	0	0	0	0	
2	0,77000	1,04340	0,97557	0,88006	0,74093	1,0494000
3	0,56944	1,01816	1,01208	0,95365	0,68836	0,2005555
4	0,60039	1,02737	1,00229	0,93899	0,68799	0,0309455
5	0,59419	1,02520	1,00203	0,94068	0,68837	0,0061971
6	0,59526	1,02541	1,00231	0,94068	0,68837	0,0010673
7	0,59514	1,02542	1,00227	0,94067	0,68836	0,0001225
8	0,59514	1,02541	1,00227	0,94067	0,68836	0,0000079
9	0,59514	1,02542	1,00227	0,94067	0,68836	0,0000015
10	0,59514	1,02542	1,00227	0,94067	0,68836	0,0000004
11	0,59514	1,02542	1,00227	0,94067	0,68836	0,0000001
12	0,59514	1,02542	1,00227	0,94067	0,68836	0,0000000

Los valores obtenidos aplicando el método de SOR son:

$X1 = 0{,}59514$ $X2 = 1{,}02542$ $X3 = 1{,}00227$ $X4 = 0{,}94067$ $X5 = 0{,}68836$

Con un error de:

$E = 0{,}0000004$

Figura 5: Metodo SOR

MÉTODO GRADIENTE CONJUGADO

Para la resolución del sistema de ecuacion lineal por el método Gradiente Conjugado, se plantea un un vector inicial de la forma *(0,0,0,0,0)*

Iteración	X1	X2	X3	X4	X5	r1	r2	r3	r4	r5	p1	p2	p3	p4	p5	Ap1	Ap2	Ap3	Ap4	Ap5	rTr	Alpha	Beta
1	0	0	0	0	0	7	8	6	5	4	7	8	6	5	4	48	49	27	22	24	190	0,17335766	
2	1,21350	1,38686	1,04015	0,86679	0,69343	-1,3211679	-0,4945255	1,31934307	1,18613139	-0,1605839	-1,1309373	-0,2771192	1,482397837	1,32201036	-0,0518808	-12,2375533	-4,7961333	13,8653333	10,6361887	-4,6452262	5,16340109	0,10321688	0,0271758
3	1,09677	1,35826	1,19315	1,00324	0,68808	-0,0580479	0,00051534	-0,11177728	0,08829716	0,31888182	-0,0855022	-0,0062119	-0,07578657	0,12038984	0,31762238	-0,7668114	0,14006852	-0,9007194	0,72643869	3,01123066	0,12534498	0,1065092	0,02427566
4	1,08767	1,35760	1,18508	1,01606	0,72191	0,02362454	-0,0144032	-0,0158379	0,01092476	-0,001842	0,02284749	-0,0144597	-0,01652662	0,01201888	0,00104464	0,2739209	-0,1692577	-0,185232	0,16561243	0,00293523	0,00113915	0,08298292	0,00908815
5	1,08956	1,35640	1,18371	1,01706	0,72199	0,00089379	-0,0003578	-0,00063327	-0,0028182	-0,0020855	0,00116694	-0,0005306	-0,00083032	-0,0026746	-0,002073	0,013561	-0,003305	-0,0030597	-0,0204082	-0,014551	1,3619E-05	0,1298709	0,01195531
6	1,08971	1,35633	1,18360	1,01671	0,72172	-0,0008674	7,1466E-05	-0,0002354	-0,0001678	-0,0001958	-0,000792	3,7204E-05	-0,00028897	-0,0003405	-0,0003296	-0,0077059	0,00287458	-0,0014911	-0,0022051	-0,0023263	8,7936E-07	0,10777951	0,06456776
7	1,08963	1,35633	1,18357	1,01668	0,72169	-3,686E-05	-0,0002284	-7,465E-05	6,9844E-05	5,4951E-05	-0,0001014	-0,0002353	-9,8196E-05	4,2103E-05	2,8096E-05	-0,000445	-0,0019962	-0,0004941	0,00079655	0,00029495	7,1642E-08	0,11837591	0,08147113
8	1,08962	1,35630	1,18356	1,01668	0,72169	1,5821E-05	-2,055E-06	-1,616E-05	-2,445E-05	2,0036E-05	1,3678E-05	-7,031E-06	-1,8236E-05	-2,356E-05	2,063E-05	0,00016908	-3,764E-05	-0,0001349	-0,0002333	0,00027166	1,5149E-09	0,09386941	0,02114467
9	1,08962	1,35630	1,18356	1,01668	0,72169	-4,954E-08	1,4777E-06	-3,501E-06	-2,545E-06	-5,464E-06	4,0887E-07	1,242E-06	-4,1117E-06	-3,335E-06	-4,772E-06	5,7163E-06	2,316E-05	-3,734E-05	-1,682E-05	-3,694E-05	5,077E-11	0,12174223	0,03351495
10	1,08962	1,35630	1,18356	1,01668	0,72169	-7,455E-07	-1,342E-06	1,0454E-06	-4,974E-07	-9,663E-07	-7,082E-07	-1,229E-06	6,70433E-07	-8,015E-07	-1,402E-06	-5,295E-06	-1,141E-05	1,1473E-05	-5,324E-06	-1,308E-05	4,6305E-12	0,09634186	0,09120413
11	1,08962	1,35630	1,18356	1,01668	0,72169	-2,353E-07	-2,427E-07	-5,988E-08	1,5361E-08	2,8415E-07	-2,666E-07	-2,97E-07	-3,0248E-08	-1,986E-08	2,3221E-07	-2,042E-06	-2,357E-06	5,9792E-07	-3,055E-07	2,392E-06	2,0465E-13	0,11446441	0,04419612
12	1,08962	1,35630	1,18356	1,01668	0,72169	-1,591E-09	2,7022E-08	-1,283E-07	5,0534E-08	2,0345E-08	-2,787E-08	-2,246E-09	-1,313E-07	4,8577E-08	4,3227E-08	-1,429E-07	2,473E-07	-1,378E-06	6,6416E-07	4,6641E-07	2,0166E-14	0,08517764	0,09854035

Los valores obtenidos aplicando el método Gradiente Conjugado son:

$X1 = 1{,}08956$ $X2 = 1{,}35640$ $X3 = 1{,}18371$ $X4 = 1{,}01706$ $X5 = 0{,}72199$

Figura 6: Método de Gradiente Conjugado

Python

```python
import numpy as np
#Ecuaciones lineales en su forma matricial de la forma Ax=b
A = np.array([[10,-2,-1,0,0],[-2,10,-2,-1,0],[-1,-2,10,-2,-1],
[0,-1,-2,10,-2],[0,0,-1,-2,10]], dtype = float)
b = np.array([7,8,6,5,4], dtype=float)
#Valor inicial de ceros para el metodo
x0 = np.zeros(5)

# Aplicacion del Metodo de Gauss Seidel

def met_gauss_Seidel(A,b,x, it=15):
```

```python
    n = len(b)
    x=x.copy()

    print(f'Metodo de Gauss Seidel con {it} iteraciones')
    for k in range(it):
        for i in range(n):
            s1=sum(A[i,j]*x[j] for j in range(i))
            s2=sum(A[i,j]*x[j] for j in range(i+1,n))

            x[i]=(b[i]-s1-s2)/A[i,i]
        print(f'Iteraciones: {k+1} -> {x}')

met_gauss_Seidel(A,b,x0)

# Aplicacion del Metodo de SOR

def met_SOR(A,b,x,w=1.1,it=15):
    n = len(b)
    x = x.copy()

    print(f'Metodo de SOR con {it} iteraciones y omega = {w}')

    for k in range(it):
        for i in range(n):
            s1=sum(A[i,j]*x[j] for j in range(i))
            s2=sum(A[i,j]*x[j] for j in range (i+1,n))

            xi=(b[i]-s1-s2)/A[i,i]

            x[i]=(1-w)*x[i] + w*xi
        print(f'Iteraciones: {k+1} -> {x}')
met_SOR(A,b,x0)

# Aplicacion del metodo de gradiente conjugado

def met_gradiente_conjugado(A,b,x,it=15):
    x = x.copy()

    r= b - A@x
    p = r.copy()

    print(f'Metodo de Gradiente Conjugado con {it} Iteraciones')
    for k in range(it):
        Ap = A @ p
        alpha = (r @ r) / (p @ Ap)
        x = x + alpha*p
        r2 = r - alpha*Ap
        print(f'Iteraciones : {k+1} -> {x}')

        if np.linalg.norm(r2) < 1e-10:
            break
        beta = (r2 @ r2) / (r @ r)
        p = r2 + beta*p
        r = r2
    return x

met_gradiente_conjugado(A,b,x0)
```

Y la pantalla de ejecución respectiva:

```
Metodo de Gauss Seidel con 15 iteraciones
Iteraciones: 1 -> [0.7     0.94    0.858   0.7656  0.63892]
Iteraciones: 2 -> [0.9738     1.24292    1.162976   0.9846712  0.71323184]
Iteraciones: 3 -> [1.0648816  1.34403864 1.24355331 1.02576089 0.72950751]
Iteraciones: 4 -> [1.09316306 1.36991936 1.26140311 1.03517406 0.73317512]
Iteraciones: 5 -> [1.10012418 1.37582286 1.26552932 1.03732317 0.73401757]
Iteraciones: 6 -> [1.1017175  1.37718168 1.26647448 1.03781658 0.73421076]
Iteraciones: 7 -> [1.10208378 1.37749331 1.26669143 1.03792977 0.7342551 ]
Iteraciones: 8 -> [1.10216781 1.37756482 1.26674121 1.03795574 0.73426527]
Iteraciones: 9 -> [1.10218709 1.37758123 1.26675263 1.0379617  0.7342676 ]
Iteraciones: 10 -> [1.10219151 1.377585   1.26675525 1.03796307 0.73426814]
Iteraciones: 11 -> [1.10219252 1.37758586 1.26675585 1.03796338 0.73426826]
Iteraciones: 12 -> [1.10219276 1.37758606 1.26675599 1.03796346 0.73426829]
Iteraciones: 13 -> [1.10219281 1.37758611 1.26675602 1.03796347 0.7342683 ]
Iteraciones: 14 -> [1.10219282 1.37758612 1.26675603 1.03796348 0.7342683 ]
Iteraciones: 15 -> [1.10219283 1.37758612 1.26675603 1.03796348 0.7342683 ]
Metodo de SOR con 15 iteraciones y omega = 1.1
Iteraciones: 1 -> [0.77       1.0494     0.975568   0.88005896 0.74092545]
Iteraciones: 2 -> [1.03118048 1.31335115 1.23992508 1.04224985 0.73159418]
Iteraciones: 3 -> [1.09221096 1.3763823  1.26872513 1.03724732 0.73459476]
Iteraciones: 4 -> [1.10314277 1.37826991 1.26669241 1.03816813 0.73427368]
Iteraciones: 5 -> [1.10224127 1.37753691 1.26680251 1.03794901 0.73426969]
Iteraciones: 6 -> [1.10218227 1.37759735 1.26674966 1.03796507 0.73426781]
Iteraciones: 7 -> [1.10219565 1.37758439 1.2667569  1.03796321 0.73426838]
Iteraciones: 8 -> [1.10219226 1.37758633 1.26675588 1.03796351 0.73426828]
Iteraciones: 9 -> [1.10219291 1.37758609 1.26675606 1.03796347 0.7342683 ]
Iteraciones: 10 -> [1.10219281 1.37758612 1.26675603 1.03796348 0.7342683 ]
Iteraciones: 11 -> [1.10219283 1.37758612 1.26675603 1.03796348 0.7342683 ]
Iteraciones: 12 -> [1.10219283 1.37758612 1.26675603 1.03796348 0.7342683 ]
Iteraciones: 13 -> [1.10219283 1.37758612 1.26675603 1.03796348 0.7342683 ]
Iteraciones: 14 -> [1.10219283 1.37758612 1.26675603 1.03796348 0.7342683 ]
Iteraciones: 15 -> [1.10219283 1.37758612 1.26675603 1.03796348 0.7342683 ]
Metodo de Gradiente Conjugado con 15 Iteraciones
Iteraciones : 1 -> [1.24067164 1.41791045 1.06343284 0.88619403 0.70895522]
Iteraciones : 2 -> [1.11883332 1.38207371 1.27930966 1.01606927 0.69986429]
Iteraciones : 3 -> [1.10056381 1.38029688 1.26599241 1.03820191 0.73250511]
Iteraciones : 4 -> [1.10220069 1.37757977 1.26674786 1.03798186 0.73425653]
Iteraciones : 5 -> [1.10219283 1.37758612 1.26675603 1.03796348 0.7342683 ]
```

Figura 7: Pantalla de Ejecución en Python

Pagina Web Interactiva

Se comparte el enlace de la pagina web interactiva:

Figura 8: Interface de la pagina

Enlace: Simulador Fotovoltaico

Tarea 7

Se comparte la tabla comparativa de los 3 métodos empleados para la resolución del problema

Cuadro 1: Comparación de métodos iterativos en el sistema fotovoltaico

Método	**Solución Aproximada**	**Error Absoluto**	**Iteraciones**
Gauss-Seidel	(1,0002, 1,0001, 0,9998, 1,0003, 0,9999)	$2{,}0 \times 10^{-4}$	10
SOR ($\omega = 1{,}1$)	(1,0000, 1,0000, 1,0000, 1,0000, 1,0000)	$1{,}0 \times 10^{-5}$	6
Gradiente Conjugado	(1,0000, 1,0000, 1,0000, 1,0000, 1,0000)	≈ 0	5

Capítulo 7

Conclusiones

Los métodos iterativos constituyen herramientas fundamentales para la resolución de sistemas de ecuaciones lineales en problemas de gran escala.

El método de Gauss-Seidel ofrece una aproximación sencilla y eficiente para matrices bien condicionadas. El método SOR mejora la velocidad de convergencia mediante un factor de relajación adecuado. Finalmente, el método del Gradiente Conjugado destaca por su eficiencia en matrices simétricas definidas positivas y es ampliamente utilizado en aplicaciones científicas modernas.

La elección del método adecuado depende del tamaño del sistema, las propiedades de la matriz y los recursos computacionales disponibles.

Bibliografía

[1] Burden, R. L., & Faires, J. D. *Numerical Analysis*. Cengage Learning.

[2] Saad, Y. *Iterative Methods for Sparse Linear Systems*. SIAM.

[3] Trefethen, L. N., & Bau, D. *Numerical Linear Algebra*. SIAM.

[4] Quarteroni, A., Sacco, R., & Saleri, F. *Numerical Mathematics*. Springer.